

Activity 3 – Promptathon .

Scenario 1 – Shopping Cart

Prompt without Framework

Weak Prompt: Test the Cart functionality

Response Received:

Cart functionality testing ▾

Test the cart functionality

Recalled 3 memories

Happy to help with this — a couple of ways I could approach "test the cart functionality" depending on what you need it for.

What kind of cart testing are you looking for? ×

- 1 Manual test cases (functional/edge cases)
- 2 Playwright automation script (TypeScript) 📄
- 3 Both — test cases + automation
- 4 I have a live site/app to test
- 📝 Something else

Skip

Prompt provided with ICEPOT framework

Act as senior QA and provide me all possible positive and negative test cases to evaluate the Cart functionality as an excel output with columns showing test scenario,expected result ,Priority and Testing status in a professional way as the scenarios could be reviewed by QC team.consider validating the add ,update and remove cart functionality and Price calculation based on the functionality [critical] and Handle out of stock functionality as well

Response Received:

I've built out **85 test cases** across two sheets:

"Cart Test Cases" — Test Case ID, Module, Test Scenario, Test Type (Positive/Negative), Expected Result, Priority (Critical/High/Medium/Low, colour-coded), Testing Status (dropdown: Not Started/In Progress/Pass/Fail/Blocked). Frozen header, autofilter, and colour-banded modules for easy QC scanning.

"Summary" — a coverage breakdown by module with formula-driven totals.

Coverage:

- **Add to Cart** — 23 cases (duplicate-item handling, stock limits, variants, guest vs. logged-in, rapid double-click, invalid product IDs)
- **Update Cart** — 15 cases (quantity steppers, manual entry, zero/negative/decimal input, stock caps, race conditions)
- **Remove from Cart** — 12 cases (empty-cart state, undo, cross-tab sync, bundle/promo impact)

- **Price Calculation** — 22 cases, marked **Critical priority** as requested (subtotal/tax/shipping/discount math, tiered pricing, negative-total guardrails, coupon edge cases, rounding, price-change-mid-cart scenarios)
- **Out of Stock Handling** — 13 cases (partial stock, checkout blocking, concurrent-purchase race condition, silent-removal prevention)

A few things worth flagging for your QC team before sign-off:

- I set default Testing Status to "Not Started" for all rows since these are new for review.
- Priority values are editable dropdowns too, in case your team wants to re-triage any of them.
- The "Add item price changes while sitting in cart" and "two users buy the last unit simultaneously" cases assume your app's actual business rule — you may want to confirm the expected behavior with your product owner and tighten that expected-result wording.



Cart_Functionality_
Test_Cases.xlsx

Observation:

1. Without Clear prompt, LLM was not able to derive a response and started to gather requirements
2. Providing Prompt ICEPOT framework, adds high value to the output as it has clear expectations and context oriented
3. Applying framework will increase the quality of the output